UNITREE G1 ELBOW CONTROL WITH DEEP Q-NETWORKS

Implementation, Training, Evaluation, and Rule-Based Comparison

Course	CSCN8020 Reinforcement Learning
Assignment	Assignment 3
Prepared by	Emmanuel Ihejiamaizu
Date	July 21, 2026
Repository	https://github.com/chooksemmanuel/Unitree_G1_DQN_Assignment

Abstract

This report presents a student-written Deep Q-Network (DQN) that controls the left elbow of a fixed-base Unitree G1 humanoid in MuJoCo. The agent receives a four-value observation, chooses among three discrete target-adjustment actions, and relies on a proportional-derivative controller with MuJoCo bias-force compensation for low-level torque control. Two epsilon-decay settings were trained headlessly for 1,000 episodes on CPU and evaluated greedily over 20 benchmark episodes. Both configurations achieved 100% success. Configuration A (decay 0.995) was selected because it produced the best mean evaluation reward (13.1796), shortest mean episode length (19.50 steps), and lowest mean final absolute error (0.0116 rad). The selected DQN also outperformed the rule-based baseline in reward and speed, although the rule-based policy used fewer action changes and required no training data. The results demonstrate that a compact DQN can learn multi-goal elbow control while preserving a clear separation between high-level reinforcement learning and conventional low-level control.

Key outcomes

Selected model	Greedy success	Mean reward	Video
Config A	**20/20**	**13.1796**	**2 min 20 s**

1. Introduction and Task Definition

The assignment extends a validated Unitree G1 primer into a complete reinforcement-learning workflow. The controlled plant is a fixed-base 29-DOF G1 model, which removes whole-body balance and locomotion from the problem so that the learning task focuses on one joint. The objective is to move the left elbow to goals sampled uniformly from -0.8 to +0.8 rad and maintain the joint within the success tolerance for several consecutive steps. Final evaluation uses four fixed goals (-0.8, -0.4, +0.4, and +0.8 rad), five episodes per goal, with epsilon fixed at 0.0.

1.1 Environment and Low-Level Control

The Gymnasium environment, G1ElbowTargetEnv, exposes a compact state and action interface while retaining physically simulated motion. The DQN does not command raw torque. Instead, it changes an internal elbow target in increments of 0.08 rad. A PD controller then tracks that target, while MuJoCo qfrc_bias compensation supplies the predictable gravity and bias torque. Non-controlled joints are held at their reset positions. This design allows the agent to learn high-level decisions rather than rediscovering basic stabilization.

Figure 1. Separation of high-level DQN decisions from low-level physical control.

1.2 Observation, Actions, Reward, and Success

Component	Implementation
Observation	[elbow angle, elbow velocity, goal angle, goal - elbow angle]
Actions	0 decrease target; 1 hold target; 2 increase target
Goal range	Uniform training goals in [-0.8, +0.8] rad
Episode limit	150 environment steps; 10 MuJoCo frames per step
Success	Absolute error <= 0.04 rad for 8 consecutive steps
Reward	-absolute error; +1 in tolerance; -0.05 for non-HOLD in tolerance; +10 at success

The reward provides dense distance information throughout the episode and a clear terminal bonus. The consecutive-step requirement prevents a policy from receiving success credit for briefly crossing the target while moving too quickly.

2. DQN Methodology

A DQN approximates the action-value function Q(s,a) with a neural network and selects the action with the largest predicted long-term return. The implementation follows the core stabilizing ideas of experience replay and a separate target network [1], [2]. All DQN components were implemented directly in PyTorch; no Stable-Baselines3 agent was used.

2.1 Network and Action Selection

Layer	Size / activation
Input	4 values
Hidden 1	64 units, ReLU
Hidden 2	64 units, ReLU
Output	3 unconstrained Q-values

Hidden-layer weights use Kaiming initialization for ReLU activations, while the output layer uses Xavier initialization. During training, epsilon-greedy selection chooses a random action with probability epsilon and otherwise chooses argmax_a Q_online(s,a). During final evaluation, greedy=True and epsilon=0.0 remove exploration completely.

2.2 Replay, Bellman Target, and Optimization

Each transition stores state, action, reward, next state, terminated, and truncated. Random mini-batches reduce temporal correlation and allow earlier experiences to be reused. Optimization begins only after at least 500 transitions are available. For a sampled transition, the selected online-network Q-value and target-network bootstrap are computed as:

Q_target = r + gamma * (1 - terminated) * max_a' Q_target_net(s', a')

Loss = SmoothL1(Q_online(s,a), Q_target)

A true termination prevents bootstrapping because the task has ended. A time-limit truncation still bootstraps because reaching the step limit does not make the next state intrinsically terminal. The target values are calculated under torch.no_grad(), gradients are clipped to a norm of 10, Adam performs the update, and the target network is synchronized every 250 optimization steps.

2.3 Hyperparameters and Controlled Comparison

Parameter	Value	Parameter	Value
Discount factor	0.95	Learning rate	0.001
Batch size	64	Replay capacity	50,000
Warm-up	500	Target update	250 steps
Epsilon start / min	1.00 / 0.05	Episode limit	150 steps
Config A decay	0.995	Config B decay	0.985
Episodes	1,000 each	Seed	42

The comparison changes only epsilon decay. Configuration A preserves exploration longer, whereas Configuration B transitions toward exploitation sooner. Keeping the network, optimizer, seed, environment, and number of episodes fixed makes the comparison interpretable.

3. Experimental Workflow and Reproducibility

The complete workflow was executed in WSL 2 Ubuntu using Python, Gymnasium 1.3.0, MuJoCo 3.10.0, NumPy 2.5.1, PyTorch 2.13.0+cpu, Matplotlib 3.11.1, and Pandas 3.0.3. Training was intentionally headless and CPU-only. The MuJoCo viewer was reserved for the final demonstration, which avoids graphical overhead and WSLg shutdown instability during training.

3.1 Validation Sequence

Stage	Evidence
Environment check	Gymnasium checker passed.
Rule-based baseline	Successful deterministic episode and 20-episode benchmark recorded.
DQN smoke test	Replay insertion, sampling, action selection, optimization, target synchronization, epsilon decay, and checkpoint loading passed.
Config A training	1,000 episodes; metrics, summary, best and final checkpoints saved.
Config B training	1,000 episodes under the same controlled conditions.
Greedy evaluation	20 episodes per DQN configuration at epsilon 0.0.
Policy comparison	Selected DQN and rule-based controller evaluated using common goals and metrics.
Rendered evidence	Selected checkpoint demonstrated at -0.8 and +0.8 rad in a 2 min 20 s video.

3.2 Reproducibility Controls

- Seed 42 was applied to Python, NumPy, PyTorch, Gymnasium resets, the action space, and replay sampling.
- Every episode used a deterministic derived seed (base seed plus episode index).
- Training and evaluation metrics were written to CSV; run summaries were written to JSON.
- Checkpoints contain online and target networks, optimizer state, epsilon, optimization count, configuration, and random states.
- The selected checkpoint can be loaded and evaluated without retraining.
- The official Unitree repository remains external and pinned to a documented revision; course-owned fixed-base XML files are submitted.

3.3 Recorded Training Scale

Metric	Config A	Config B
Replay transitions	24,550	20,806
Optimization steps	24,051	20,307
Wall-clock time	194.74 s	144.22 s
Time relative to A	100.0%	74.1%

Configuration B generated fewer transitions and optimization steps because successful episodes became shorter earlier. It therefore finished 50.52 seconds faster, a 25.9% reduction from Configuration A, while remaining far below the five-hour limit.

4. Training Results

Both agents began with unstable, highly negative returns because random actions frequently moved the internal target away from the goal and episodes reached the time limit. As useful transitions accumulated, the moving-average reward increased rapidly and then stabilized in a narrow positive range. The remaining raw variation is expected because goals are sampled across different travel distances.

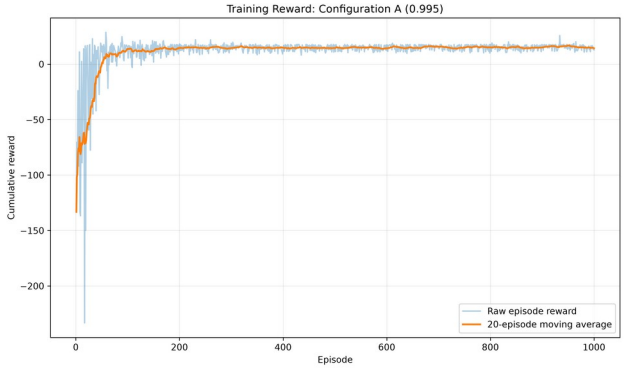

Figure 2. Configuration A raw reward and 20-episode moving average.

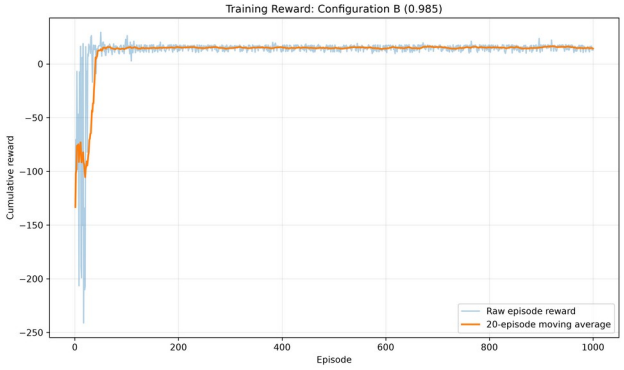

Figure 3. Configuration B raw reward and 20-episode moving average.

4.1 Quantitative Training Comparison

Metric	Config A (0.995)	Config B (0.985)
Episodes	1,000	1,000
Training time	194.74 s	144.22 s
Final epsilon	0.05	0.05
Final-20 mean reward	14.3615	14.3823
Final-50 success rate	100%	100%
First 80% rolling success	Episode 74	Episode 62
First 100% rolling success	Episode 112	Episode 84
Epsilon first logged at 0.05	Episode 599	Episode 200

Configuration B learned faster in the early phase: its 50-episode rolling success first reached 80% at episode 62 and 100% at episode 84, compared with episodes 74 and 112 for Configuration A. The faster schedule reached the minimum epsilon near episode 200, whereas Configuration A continued meaningful exploration until roughly episode 599. By the end, both were equally successful and their final mean rewards differed by only 0.0209.

5. Exploration, Success, and Optimization Behaviour

Figure 4. Rolling training success; both configurations exceed the required 80% threshold.

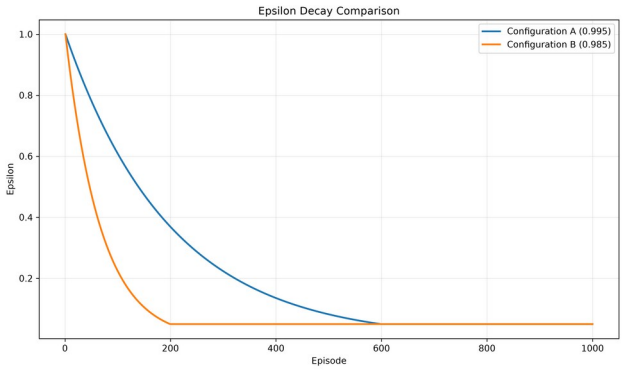

Figure 5. Controlled difference in the two exploration schedules.

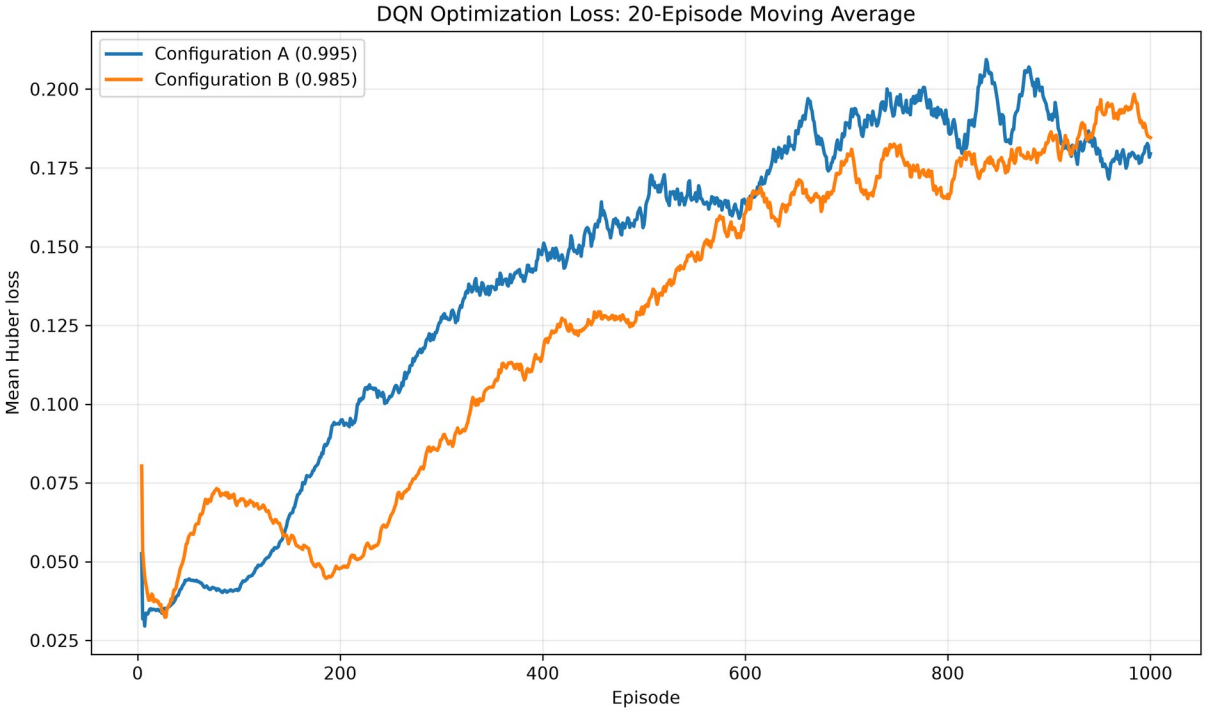

Figure 6. Twenty-episode moving average of Huber optimization loss.

5.1 Interpretation

The success curves show that both agents crossed the 80% requirement early and then remained at 100% for most of training. Configuration B benefited from earlier exploitation because the task is low-dimensional, deterministic, and supported by a stable low-level controller. Configuration A collected more exploratory transitions, which slightly improved its final greedy evaluation.

The Huber loss does not decrease monotonically. Its moving average rises as the replay buffer contains more successful terminal transitions and the learned Q-value scale increases. This is not evidence of divergence by itself: reward, success, episode length, and greedy evaluation all stabilized. The loss is therefore interpreted together with control performance rather than as a standalone supervised-learning metric.

6. Greedy Evaluation and Model Selection

Both trained configurations were evaluated at epsilon 0.0 for five episodes at each required goal. Each configuration achieved 20 successes from 20 episodes, exceeding the 80% requirement. The closer goals required 17 steps on average, while the extreme goals required approximately 22 steps because the elbow had farther to travel from the zero reset state.

Goal	Episodes	Successes	Rate	Mean reward	Mean steps	Mean error
-0.8 rad	5	5	100%	10.9015	22.0	0.0011
-0.4 rad	5	5	100%	15.4933	17.0	0.0124
+0.4 rad	5	5	100%	15.4339	17.0	0.0161
+0.8 rad	5	5	100%	10.8899	22.0	0.0170
Overall	20	20	100%	13.1796	19.50	0.0116

Table 1. Required 20-episode greedy evaluation for selected Configuration A.

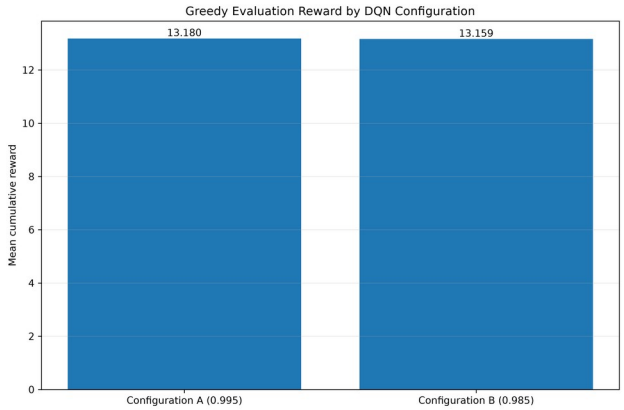

Figure 7. Mean greedy evaluation reward for the two trained configurations.

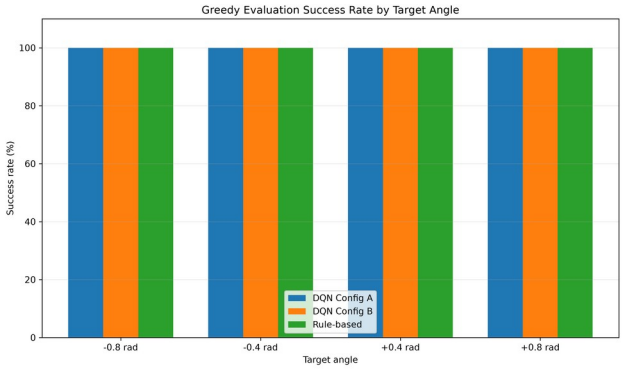

Figure 8. Success rate by benchmark target angle.

6.1 Selection Recommendation

Metric	Config A	Config B	Preferred
Success rate	100%	100%	Tie
Mean evaluation reward	13.1796	13.1588	A
Mean episode length	19.50	19.75	A
Mean final absolute error	0.0116	0.0127	A
Training time	194.74 s	144.22 s	B

Configuration A was selected and copied to models/selected_dqn.pt. The performance differences are small, but A is consistently better on all three final quality metrics: reward, completion speed, and final error. Configuration B remains the better efficiency choice when shorter training time is more important than a marginal evaluation advantage.

7. Rule-Based Baseline versus Selected DQN

The rule-based controller and selected DQN were evaluated on the same four goals and 20-episode structure. Both achieved perfect success, but their behaviour differed. The rule-based policy uses domain knowledge to move toward the goal and then hold. The DQN learns a more aggressive sequence of corrections that reaches success sooner but changes action more often.

Metric	Rule-based	Selected DQN (A)
Successes / 20	20 / 20	20 / 20
Success rate	100%	100%
Mean cumulative reward	12.8666	13.1796
Mean episode length	24.00	19.50
Mean final absolute error	0.0122	0.0116
Mean HOLD actions	16.50	4.75
Mean action changes	1.00	6.50
Training samples	None	24,550 transitions

Table 2. Fair comparison using the same benchmark goals and evaluation metrics.

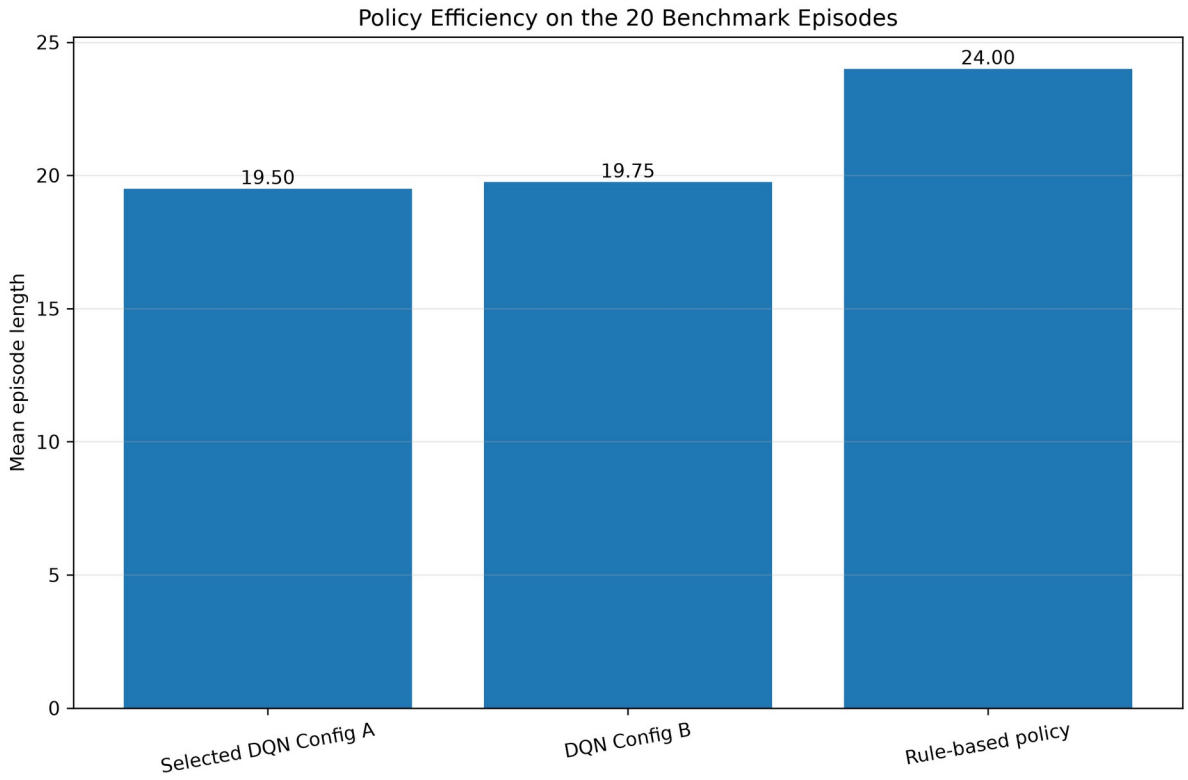

Figure 9. The selected DQN completes the benchmark episodes faster than the rule-based baseline.

7.1 Behavioural Analysis

- Sample efficiency: the rule-based policy requires no data collection or optimization.
- Stability: one action change and 16.5 HOLD actions make the rule-based behaviour smoother and easier to interpret.
- Generalization: the DQN achieved 100% success across both signs and both travel distances.
- Use of HOLD: 4.75 HOLD actions per episode show that the DQN learned HOLD as a useful decision.
- Oscillation: 6.5 action changes indicate extra corrective switching, but final errors remained low.
- Simple-task advantage: rules can encode the correct direction directly; RL becomes more valuable as dynamics and objectives become harder to specify.

8. Limitations, Safety, and Conclusion

8.1 Limitations

- The environment is fixed-base and controls one joint; the result does not demonstrate humanoid balance, locomotion, or coordinated whole-body motion.
- Final evaluation starts from the same zero state without external disturbances. Because the simulator is deterministic, the five episodes at each fixed goal produce identical behaviour; robustness to noise and perturbations remains untested.
- Only one random seed and two epsilon schedules were studied. Repeated training seeds would provide stronger evidence about variance and reproducibility of learning outcomes.
- The action set changes an intermediate target by a fixed 0.08 rad, so performance depends partly on the approved PD controller and reward design.
- The DQN changes actions more frequently than the rule-based controller, suggesting that an action-change penalty or a richer state representation could improve smoothness.

8.2 Physical-Robot Safety

This work is a simulation-only result. The trained checkpoint must not be transferred directly to a physical Unitree G1. Hardware use would require validated joint and torque limits, emergency-stop procedures, supervised low-power testing, collision controls, latency analysis, calibration, and additional safety engineering.

8.3 Conclusion

The project successfully implemented the required DQN components, completed both controlled epsilon-decay experiments, and produced reproducible training evidence, checkpoints, plots, greedy evaluation, rule-based comparison, and a rendered demonstration. Both DQN configurations achieved 100% final success. Configuration B learned faster and reduced training time, while Configuration A delivered the strongest final benchmark metrics and was selected for submission. Compared with the rule-based baseline, the DQN required training and used more action changes, but achieved higher reward, shorter episodes, and slightly lower final error. The main technical result is therefore not only that the elbow reached its targets, but that a saved, student-written DQN transformed observations and goals into effective action values across the full approved multi-goal range.

AI-Assistance Disclosure

Generative AI assistance was used to explain reinforcement-learning and MuJoCo concepts, review implementation structure, debug commands and runtime errors, suggest validation tests, organize experimental results, and assist with README and report drafting. All commands were executed in the student's own environment. The student ran and verified the environment tests, training, evaluation, checkpoint loading, plots, simulation, and final video, and remains responsible for understanding and submitting the work.

References

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.

[2] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, pp. 529-533, 2015.

[3] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in Proc. IEEE/RSJ IROS, 2012.

[4] PyTorch, Gymnasium, and MuJoCo official software documentation, used for the submitted implementation environment.

[5] CSCN8020 Reinforcement Learning, Unitree G1 DQN Assignment specification, 2026.